Taller número 3. Programación Numérica con Python y NumPy.

Resultado de aprendizaje:	• **RAA1.** Aplica conceptos de álgebra vectorial y transformaciones geométricas para representar objetos y procesar imágenes digitales en entornos bidimensionales. *(RAP1, RAP3)* • **RAA4.** Implementa videojuegos funcionales como proyectos integradores, incorporando simulación física, animación y estructuras de control gráfico en tiempo real. (RAP1, RAP3, RAP4, RAP5, RAP7) • **RAA5.** Comunica, documenta y presenta desarrollos gráficos utilizando lenguaje técnico apropiado, argumentación estructurada y herramientas de soporte. (RAP8)
Valor de calificación:	Entre 0.0 y 5.0
Tiempo estimado de desarrollo:	4 h
Dinámica de la actividad:	• Taller individual • Ejercicios distribuidos en diferentes subrutinas • Subida del archivo en formato .py a la plataforma
Producto a entregar:	Código fuente del taller en formato py
Documentos de Apoyo:	Ver material de análisis y lectura de la unidad
Herramientas:	Compilador python
Instrumento de evaluación	Rubrica de evaluación para talleres
Fecha de apertura:	
Fecha de cierre:	

Descripción: Este taller está diseñado para introducir y afianzar el uso de la biblioteca NumPy en Python, con el objetivo de desarrollar habilidades en la creación y manipulación de arrays numéricos, realizar operaciones matemáticas básicas y complejas, y gestionar datos con Python para aplicaciones en ciencia de datos, ingeniería y matemáticas.

Objetivos del Taller:

- Familiarizar a los participantes con la creación y manipulación de arrays en NumPy.
- Desarrollar habilidades para realizar operaciones matemáticas básicas con arrays.

- Aplicar técnicas de indexación y slicing para el manejo efectivo de subconjuntos de datos.
- Utilizar broadcasting y funciones universales para simplificar operaciones matemáticas.
- Aprender métodos de manipulación de formas y álgebra lineal en matrices.
- Manejar y tratar datos faltantes dentro de arrays numéricos.
- Practicar el almacenamiento y carga de arrays para su uso en análisis de datos.

Metodología:

- Se combinarán breves sesiones teóricas con prácticas guiadas en computadora.
- Los ejercicios se resolverán paso a paso, con explicaciones detalladas y asistencia individualizada.
- Se promoverá la discusión grupal para explorar diferentes soluciones y enfoques.

Recursos:

- Computadoras con Python y NumPy instalados.
- Acceso a documentación en línea de NumPy y otros recursos educativos.
- Cuadernos de trabajo o espacios de codificación en línea como Jupyter Notebooks.

Evaluación:

- Se revisará la corrección de los ejercicios individuales.
- Se valorará la capacidad para integrar los conceptos aprendidos en un programa final con menú interactivo.
- Se alentará a los participantes a explicar su código y el razonamiento detrás de sus soluciones.

Contenido del Taller:

- **Ejercicio 1: Creación y Propiedades de Arrays**

1. Cree un array unidimensional con los números del 1 al 10.
2. Cambie la forma del array para que tenga dimensión 2x5.
3. Imprima en pantalla:
    - El array original.
    - El array reestructurado.
    - Su forma (`shape`).
    - Su tamaño total (`size`).
    - Su número de dimensiones (`ndim`).

Requisitos

- Debe utilizar `np.array()` y `.reshape()`.

- Toda la salida debe mostrarse con mensajes descriptivos (no solo imprimir el valor).

- **Ejercicio 2: Operaciones Básicas entre Arrays**

1. Cree dos arrays `a` y `b` con 5 valores numéricos cada uno.
2. Realice las siguientes operaciones:
    - Suma elemento a elemento.
    - Resta elemento a elemento.
    - Producto elemento a elemento.
    - Suma total de los elementos de `a`.
3. Muestre todos los resultados en pantalla con descripción clara.

Requisitos

- No usar ciclos `for`; debe usar operaciones vectorizadas de NumPy.
- Explicar en un comentario qué significa "operación elemento a elemento".

Ejercicio 3: Indexación y Slicing

1. Cree un array con los números del 0 al 19.
2. Imprima:
    - El quinto elemento.
    - Los elementos desde la posición 2 hasta la 6.
    - Los últimos tres elementos.
3. Modifique el elemento en la posición 0 por el valor 100.
4. Muestre el array actualizado.

Requisitos

- Utilizar indexación directa y slicing.
- Incluir comentarios explicando cómo funciona el slicing.

Ejercicio 4: Broadcasting y Funciones Universales (ufunc)

1. Cree una matriz 3x3 con valores del 1 al 9.
2. Sume el número 10 a toda la matriz usando broadcasting.
3. Calcule la raíz cuadrada de cada elemento.
4. Imprima los resultados explicando qué es broadcasting.

Requisitos

- No usar ciclos.
- Utilizar al menos una función universal de NumPy (`np.sqrt`, `np.exp`, etc.).

Ejercicio 5: Manipulación de Formas y Álgebra Lineal

1. Cree un array con 6 números.
2. Reestructúrelo en una matriz de 3x2.
3. Calcule el producto punto entre la matriz y su transpuesta.
4. Muestre el resultado e indique qué dimensiones tiene la matriz final.

Requisitos

- Utilizar `.reshape()` y `np.dot()` o el operador `@`.
- Explicar en un comentario qué representa el producto punto en matrices.

Ejercicio 6: Manejo de Datos Faltantes

1. Cree un array que contenga al menos dos valores `np.nan`.
2. Reemplace los valores `nan` por 0.
3. Calcule:
    - La media del array original.
    - La media del array corregido.
4. Compare los resultados y explique la diferencia en un comentario.

Requisitos

- Utilizar `np.nan_to_num()` o funciones similares.
- Mostrar claramente los resultados antes y después del reemplazo.

Ejercicio 7: Guardar y Cargar Arrays

1. Cree un array con valores numéricos.
2. Guárdelo en un archivo llamado `datos.npy`.
3. Cargue el archivo nuevamente en otra variable.
4. Verifique que los datos cargados sean iguales a los originales.

Requisitos

- Utilizar `np.save()` y `np.load()`.
- Mostrar en pantalla el resultado de la verificación.

Programa Integrador con Menú de Opciones: Desarrollar un programa que incorpore todos los ejercicios anteriores en un menú interactivo, reforzando así las habilidades de programación y consolidando el conocimiento adquirido.

Rúbrica de Evaluación

Criterio	**RAA Evaluado**	**Excelente (4.5 – 5.0)**	**Bueno (3.5 – 4.4)**	**Aceptable (2.5 – 3.4)**	**Insuficiente (0.0 – 2.4)**
1. Análisis del problema físico	RAA1	Interpreta correctamente el fenómeno físico y lo traduce adecuadamente al modelo computacional.	Interpreta el fenómeno con pocas fallas conceptuales.	El análisis es limitado o parcialmente incorrecto.	No hay un análisis claro del fenómeno físico.
2. Aplicación de transformaciones	RAA1	Emplea correctamente fórmulas y estructuras gráficas para simular la física.	Aplica transformaciones con mínimos errores.	Uso confuso o incompleto de las transformaciones.	No aplica transformaciones o son incorrectas.
3. Producto funcional con menú	RAA4	El programa incluye todas las funciones integradas y permite navegación efectiva.	Integra la mayoría de funciones, con buen manejo de opciones.	Integración parcial o con errores de navegación.	Menú inexistente o con errores críticos.
4. Organización y modularidad	RAA1	Código estructurado, con funciones, nombres claros y comentarios útiles.	Estructura mayormente clara con uso razonable de funciones.	Funcional, pero con escasa claridad o sin modularización.	Código desordenado y sin estructura funcional.

5. Uso de herramientas Python	RAA1	Utiliza eficientemente estructuras de control, funciones y librerías pertinentes.	Uso adecuado de herramientas básicas.	Herramientas mal utilizadas o subutilizadas.	Herramientas inapropiadas o ausentes.
6. Entrega y documentación	RAA5	Archivo entregado puntualmente, bien nombrado y documentado.	Archivo correcto y entregado a tiempo.	Entregado con errores de forma o retraso.	No entrega o sin documentación mínima.

Ponderación sugerida por criterio

Criterio	**Peso (%)**
Análisis del problema físico	20%
Aplicación de transformaciones	20%
Producto funcional con menú	20%
Organización y modularidad	15%
Uso de herramientas Python	15%
Entrega y documentación	10%
Total	**100%**

Elaborado por:
FRANCISCO ALEJANDRO MEDINA